

Week 6

Create a client-server application using Named Pipes (FIFOs). The client sends messages and the server processes and responds to them. Analyze FIFO behavior when multiple clients communicate simultaneously.

```
GNU nano 7.2 server.c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/stat.h>
#include <sys/wait.h>
#define SERVER_FIFO "/tmp/server_fifo"
int main() {
    char buffer[100];
    // Create server FIFO
    mkfifo(SERVER_FIFO, 0666);

    printf("Server started...\n");

    while (1) {
        int fd = open(SERVER_FIFO, O_RDONLY);

        if (fd == -1) {
            perror("open");
            exit(1);
        }

        // Read message
        ssize_t bytes_read = read(fd, buffer, sizeof(buffer) - 1);
        close(fd);
    }
}
```

```
buffer[bytes_read] = '\0';

printf("Message received: %s\n", buffer);

// Create a child process to handle each client
if (fork() == 0) {
    char response[120];

    printf("Processing client request...\n");

    snprintf(response, sizeof(response),
             "Server received: %s", buffer);

    // Print response
    printf("Response: %s\n", response);

    exit(0);
}

// Check for completed child processes
waitpid(-1, NULL, WNOHANG);
}

unlink(SERVER_FIFO);

return 0;
}
```

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <unistd.h>

#define SERVER_FIFO "/tmp/server_fifo"

int main() {
    char message[100];

    printf("Enter message: ");
    fgets(message, sizeof(message), stdin);

    int fd = open(SERVER_FIFO, O_WRONLY);

    if (fd == -1) {
        perror("open");
        exit(1);
    }

    write(fd, message, strlen(message) + 1);

    close(fd);

    printf("Message sent to server.\n");

    return 0;
}

```

OUTPUT:

```

root@Tejasri:~# ./client
Enter message: Hiii
Message sent to server.
root@Tejasri:~# |

```

```

root@Tejasri:~# ./server
Server started...
Message received: Hiii

Processing client request...
Response: Server received: Hiii

```

Task 2

Create a POSIX signal handling program that captures SIGINT, SIGTERM, and SIGUSR1. Demonstrate asynchronous event handling and explain the role of signal handlers.

```
#include <stdio.h>
#include <stdlib.h>
#include <signal.h>
#include <unistd.h>

void signal_handler(int sig) {

    if (sig == SIGINT) {
        printf("\nSIGINT received!\n");
        printf("This is usually generated by pressing Ctrl+C.\n");
    }

    else if (sig == SIGTERM) {
        printf("\nSIGTERM received!\n");
        printf("Termination request received.\n");
    }

    else if (sig == SIGUSR1) {
        printf("\nSIGUSR1 received!\n");
        printf("User-defined signal received.\n");
    }

}

int main() {

    // Register signal handlers
    signal(SIGINT, signal_handler);
    signal(SIGTERM, signal_handler);
    signal(SIGUSR1, signal_handler);
```

```
    printf("Waiting for signals...\n");

    while (1) {
        printf("Program is running...\n");
        sleep(3);
    }

    return 0;
}
```

OUTPUT:

```
root@Tejasri:~# nano signals.c
root@Tejasri:~# gcc signals.c -o signals
root@Tejasri:~# ./signals
Process ID: 625
```

[illegible][illegible][illegible][illegible][illegible][illegible][illegible][illegible][illegible][illegible][illegible][illegible][illegible][illegible][illegible][illegible][illegible][illegible]

```
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
```

```
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
```

```
Program is running...
Program is running...
Program is running...
Program is running...
```

```
Program is running...
Program is running...
Program is running...
```

```
Program is running...
Program is running...
```